

AI-ASSIGEMENT-13.4

NAME:-NIPUN.I

H.T.NO:-2303A52208

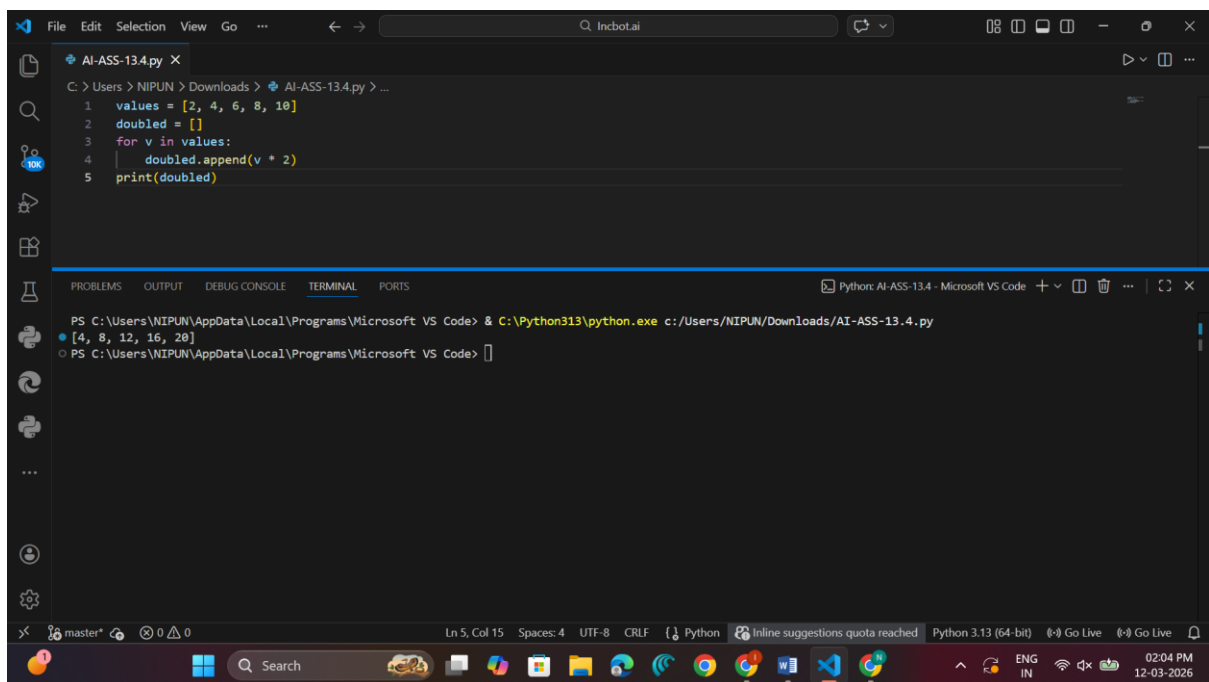
Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions Week 7

Task 1 – Refactoring Data Transformation

Prompt:

I have a Python list of numbers and a for-loop that doubles each value and appends it to a new list. How can I refactor this to be more Python and efficient using a list comprehension?

Code & Output:



The screenshot shows a Visual Studio Code editor window with a Python file named 'AI-ASS-13.4.py'. The code in the file is as follows:

```
1 values = [2, 4, 6, 8, 10]
2 doubled = []
3 for v in values:
4     doubled.append(v * 2)
5 print(doubled)
```

Below the code editor, the terminal window shows the command to run the script and its output:

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-13.4.py
[4, 8, 12, 16, 20]
```

The status bar at the bottom of the editor indicates the current line and column (Ln 5, Col 15), the file encoding (UTF-8), the line ending (CRLF), and the active language (Python). It also shows a message about the inline suggestions quota being reached.

Observation:

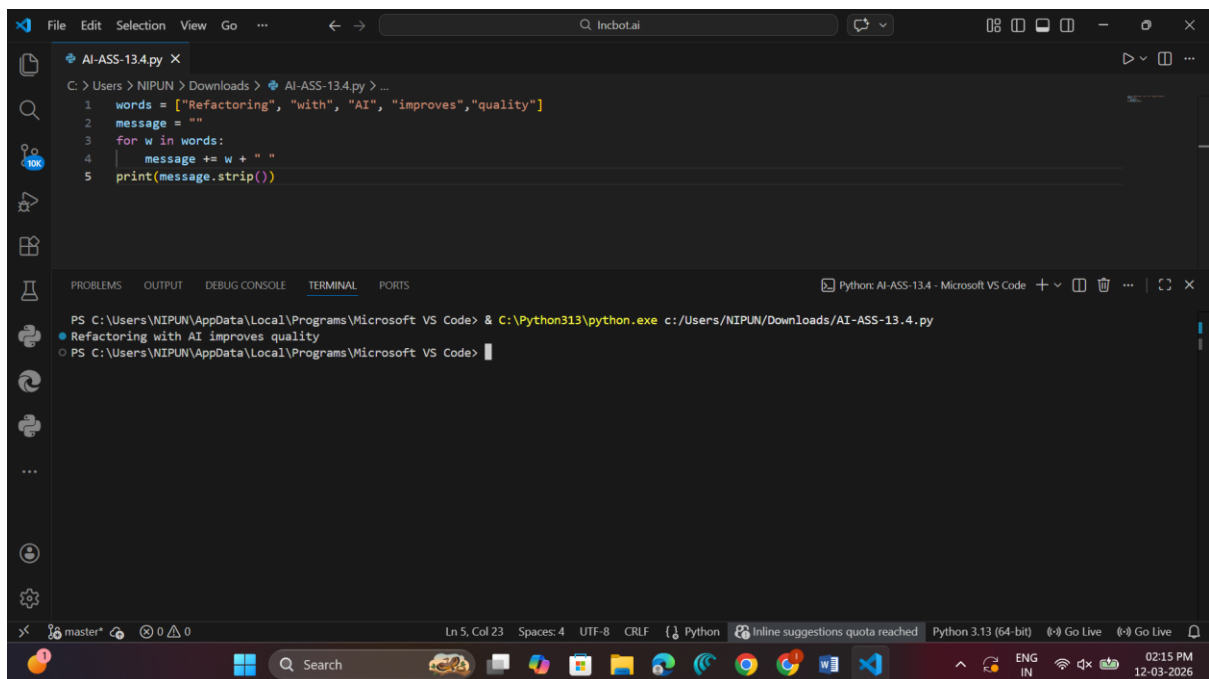
The original code used a loop and `append()` method to add elements to a list. After refactoring, **list comprehension** was used, which makes the code shorter, cleaner, and more readable while producing the same output.

Task 2 – Improving Text Processing Code

Prompt:

Suggest a more efficient and readable way to construct a sentence from a list of words instead of repeatedly concatenating strings inside a loop.

Code & Output:



The screenshot shows the Microsoft Visual Studio Code editor interface. The editor window displays a Python file named `AI-ASS-13.4.py` with the following code:

```
1 words = ["Refactoring", "with", "AI", "improves", "quality"]
2 message = ""
3 for w in words:
4     message += w + " "
5 print(message.strip())
```

The terminal window at the bottom shows the command `python c:/Users/NIPUN/Downloads/AI-ASS-13.4.py` being executed, resulting in the output: `Refactoring with AI improves quality`.

Observation:

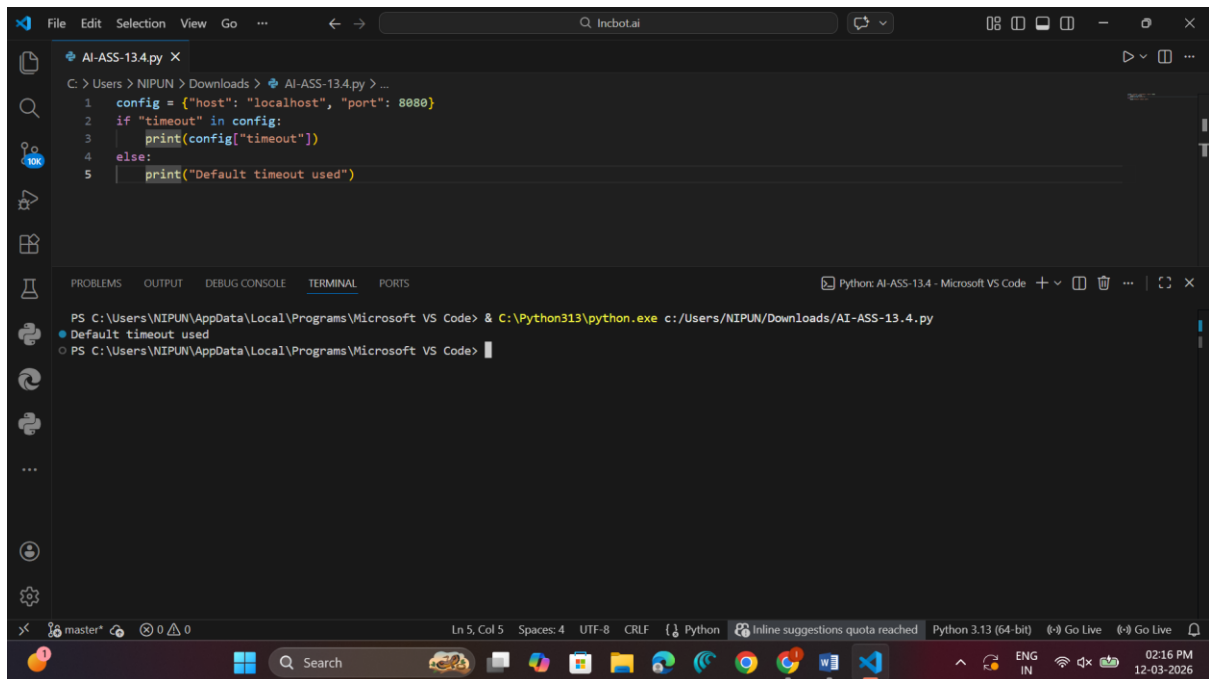
The legacy code used repeated string concatenation, which can be inefficient. The refactored version uses the `join()` method, which combines all words in the list with a space separator, making the code simpler and more efficient.

Task 3 – Safer Access to Configuration Data

Prompt:

Refactor the code that manually checks for dictionary keys by using a safer dictionary access method in Python.

Code & Output:



The screenshot shows a Visual Studio Code editor window with a Python file named `AI-ASS-13.4.py`. The code defines a `config` dictionary with `host` and `port` keys. It uses an `if` statement to check if the `timeout` key exists in the dictionary. If it exists, it prints the value; otherwise, it prints a default message. The terminal output shows the command being executed and the resulting output: `Default timeout used`.

```
1 config = {"host": "localhost", "port": 8080}
2 if "timeout" in config:
3     print(config["timeout"])
4 else:
5     print("Default timeout used")
```

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-13.4.py
Default timeout used
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation:

Instead of manually checking if a key exists in the dictionary, the `get()` method is used. This method safely retrieves the value if the key exists and returns a default value if the key is missing, making the code more concise and reliable.

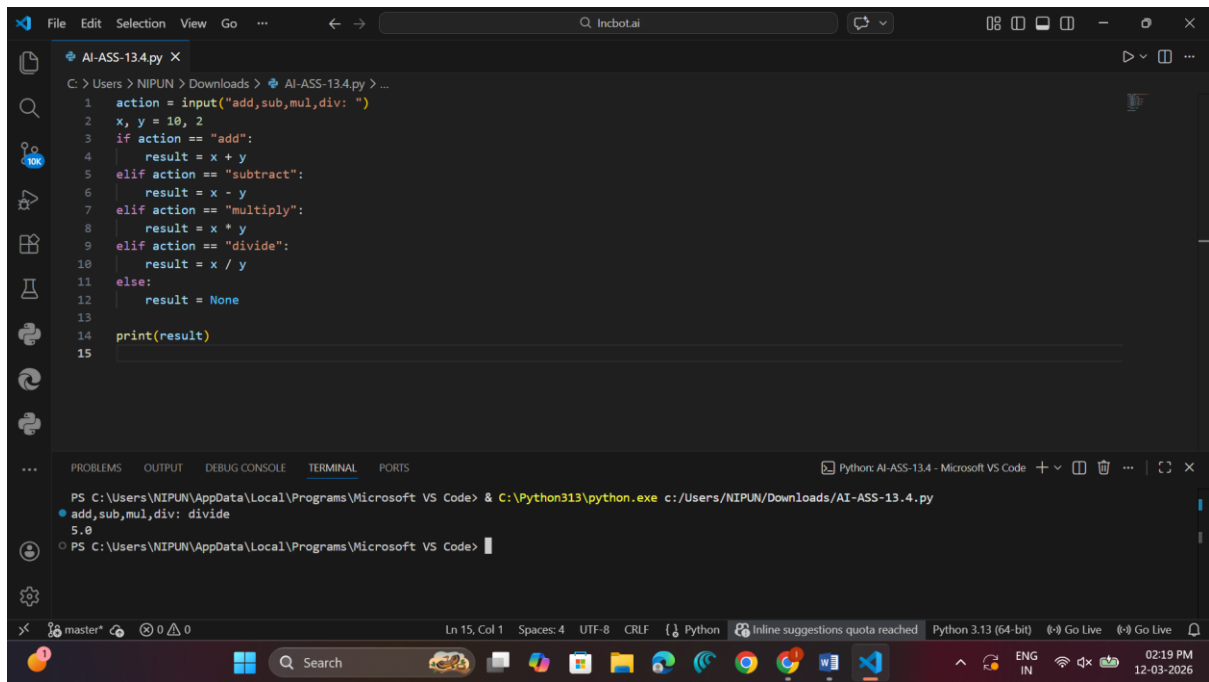
=====

Task 4 – Refactoring Conditional Logic

Prompt:

Replace multiple `if-elif` conditions used for different operations with a cleaner and more scalable mapping technique.

Code & Output:



The screenshot shows a VS Code editor window with a Python file named `AI-ASS-13.4.py`. The code is a simple calculator that takes an action and two numbers as input and performs the corresponding operation. The terminal shows the command `python c:/Users/NIPUN/Downloads/AI-ASS-13.4.py` being executed, and the output is `5.0`.

```
1 action = input("add,sub,mul,div: ")
2 x, y = 10, 2
3 if action == "add":
4     result = x + y
5 elif action == "subtract":
6     result = x - y
7 elif action == "multiply":
8     result = x * y
9 elif action == "divide":
10    result = x / y
11 else:
12    result = None
13
14 print(result)
15
```

Terminal Output:

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-13.4.py
add,sub,mul,div: divide
5.0
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation:

The original code used many `if-elif` statements to perform operations. The refactored code uses a **dictionary mapping approach**, which improves readability, reduces code length, and makes it easier to add new operations in the future.

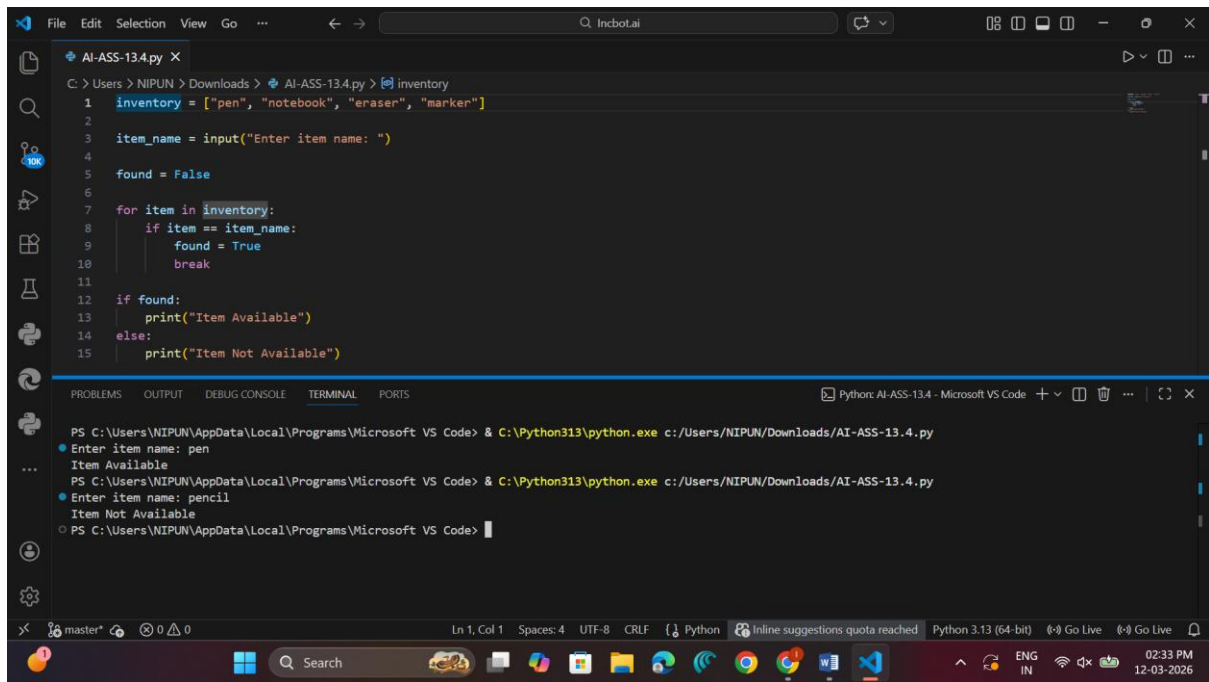
=====

Task 5 – Simplifying Search Logic

Prompt:

Refactor the manual loop used to search for an item in a list by using a more concise Pythonic approach.

Code & Output:



The screenshot shows a Visual Studio Code editor window with a Python file named `AI-ASS-13.4.py`. The code defines a list `inventory` containing `"pen", "notebook", "eraser", "marker"`. It prompts the user to enter an item name and checks if it exists in the `inventory` list using a `for` loop and a `found` flag variable. The terminal output shows two successful runs: one for `pen` (Item Available) and one for `pencil` (Item Not Available).

```
1 inventory = ["pen", "notebook", "eraser", "marker"]
2
3 item_name = input("Enter item name: ")
4
5 found = False
6
7 for item in inventory:
8     if item == item_name:
9         found = True
10        break
11
12 if found:
13     print("Item Available")
14 else:
15     print("Item Not Available")
```

Terminal Output:

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-13.4.py
Enter item name: pen
Item Available
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-13.4.py
Enter item name: pencil
Item Not Available
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation:

The legacy code used a loop and a flag variable to check if an item existed in the list. The refactored code uses the **in operator**, which directly checks for membership in the list and makes the code shorter and easier to understand.